

Les curseurs (1/22)

- Dans un bloc PL/SQL, le langage PL/SQL prend en charge les instructions LMD (pour extraire et modifier des données de la table de base de données) ainsi que les commandes de gestion des transactions (commit, rollback).
- L'utilisation du résultat de la requête se fait à travers la commande INTO

Les curseurs (2/22)

- Il faut définir autant de variables dans la clause `INTO` que de colonnes de base de données dans la clause `SELECT` en s'assurant qu'elles correspondent de manière appropriée et que les types de données sont compatibles.
- Les fonctions de groupe, telles que `SUM`, `COUNT`, ... dans une instruction SQL peuvent être utilisées puisqu'elles s'appliquent à des ensembles de lignes dans une table

Les curseurs (3/22)

Soit la requête suivante

Declare

Nompers personne.nomp%type;

Salairepers personne.salairep%type;

Postepers personne.postep%type;

Begin

SELECT nomp, postep, salairep into

nompers, postepers, salairepers

from personne where numservp = 20;

end;

/

- Les instructions de type **SELECT ... INTO ...** manquent de souplesse, elles ne fonctionnent que sur des requêtes retournant une et une seule valeur.
- Ne serait-il donc pas intéressant de pouvoir placer dans des variables le résultat d'une requête retournant plusieurs lignes ?

Les curseurs (4/22)

- Etant donné que les requêtes renvoient très souvent un nombre important et non prévisible de lignes.
- On introduit, donc une notion de “curseur” pour récupérer (et exploiter) les résultats de requêtes.
- Un curseur est une zone mémoire de taille fixe, utilisée par le moteur SQL pour analyser et interpréter un ordre SQL
- Il contient le résultat d'une requête (0, 1 ou plusieurs lignes).

Les curseurs (5/22)

Il existe deux types de curseurs :

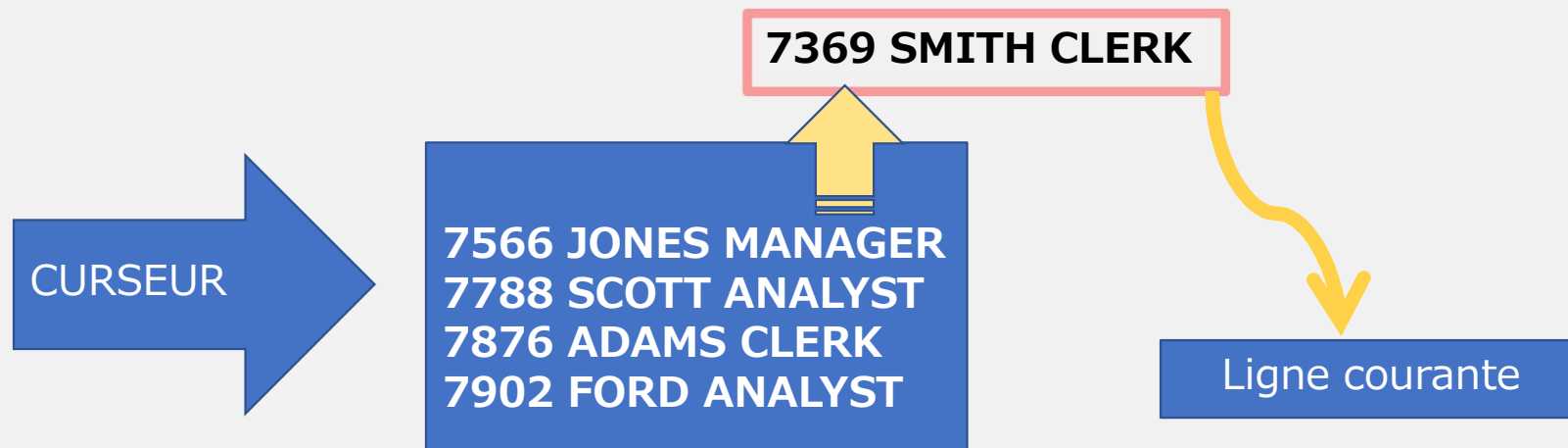
Curseurs implicites : créés et gérés en interne par le serveur Oracle afin de traiter les instructions SQL

Curseurs explicites : déclarés explicitement par le programmeur

Les curseurs (6/22)

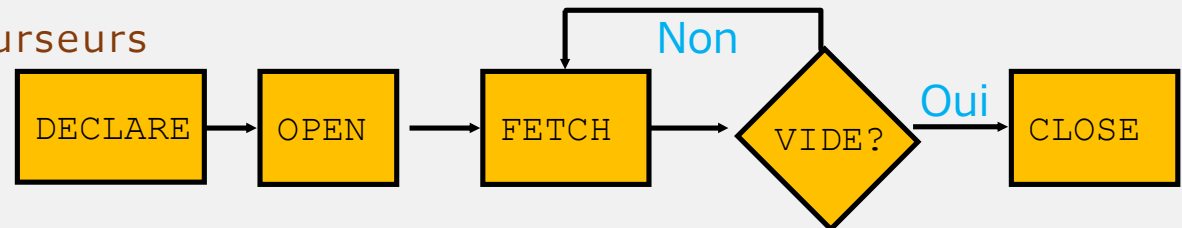
Les curseurs explicites

Un curseur explicite, contrairement au curseur implicite est géré par l'utilisateur pour traiter un ordre Select qui ramène plusieurs lignes



Les curseurs (7/22)

Etapes d'utilisation des curseurs



Déclarer le curseur dans la section déclarative d'un bloc PL/SQL en le nommant et en définissant la structure de l'interrogation à y associer.

Ouvrir le curseur: L'instruction **OPEN** exécute l'interrogation et attache toutes les variables référencées. Les lignes identifiées par l'interrogation constituent l'ensemble actif et peuvent désormais être extraites (**FETCH**).

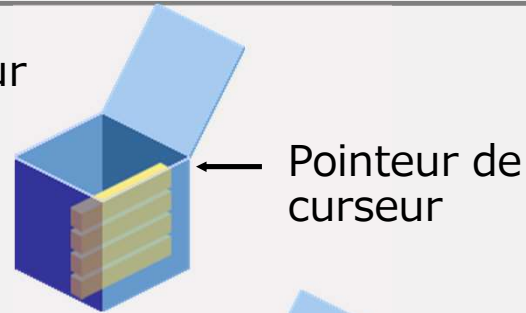
Procéder à l'extraction (FETCH) des données à partir du curseur. Dans le diagramme de flux présenté dans la diapositive ci-dessus, après chaque extraction (fetch), vous testez l'existence de la ligne dans le curseur. S'il n'y a plus de lignes à traiter, vous devez fermer le curseur.

Fermer le curseur. L'instruction **CLOSE** libère l'ensemble actif de lignes. Il est désormais possible de rouvrir le curseur pour établir un nouvel ensemble actif.

Les curseurs (8/22)

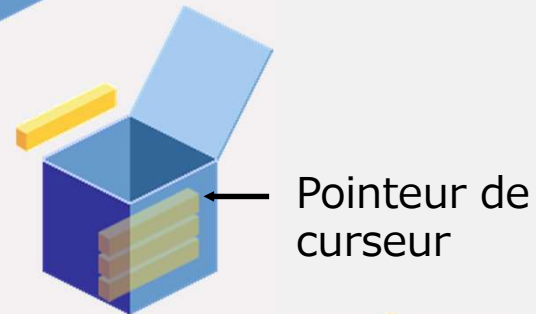
1

Ouverture du curseur



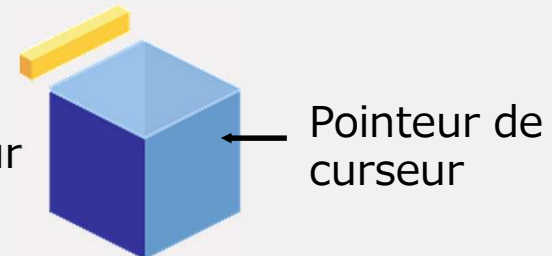
2

Extraction (fetch)
d'une ligne



3

Fermeture du curseur



Les curseurs (9/22)

Syntaxe

- *Déclaration du curseur*

CURSOR nomcurseur IS requête ;

- *Exemple*

```
DECLARE
    CURSOR curseur_pers IS
    SELECT nump, nomp
    FROM personne
    WHERE numservp=20;

    numero  personne.nump%type;
    nom      personne.nomp%type;
```

Les curseurs (10/22)

- Ouverture du curseur

Begin

OPEN curseur_pers;

- Extraction des lignes

LOOP

FETCH curseur_pers INTO numero, nom;

EXIT WHEN curseur_pers%NOTFOUND;

*DBMS_OUTPUT.PUT_LINE('le numero est:' || numero
|| ' le nom est' || nom);*

END LOOP;

- Fermeture du curseur

CLOSE curseur_pers;

Les curseurs (11/22)

Exemple

Sélectionner l'ensemble des employés (numéro, nom et salaire) dont le salaire ne dépasse pas 400 dinars et les augmenter de 5 dinars.

Les curseurs (12/22)

Curseurs et boucle FOR

- Il existe une boucle FOR se chargeant de l'ouverture, de la lecture des lignes du curseur et de sa fermeture.
- Elle simplifie le traitement des curseurs explicites.
- Des opérations d'ouverture, d'extraction (fetch), de sortie et de fermeture ont lieu de manière implicite.
- L'enregistrement est déclaré implicitement.

Les curseurs (12/22)

Curseurs et boucle FOR

Syntaxe

```
FOR nom_enregistrement IN nom_curseur LOOP  
    instruction1;  
    instruction2;  
    . . .  
END LOOP;
```

– *Rq!* Le nom de l'enregistrement est déclaré implicitement

Exemple

Sélectionner l'ensemble des employés dont le salaire dépasse 1000 dinars et les afficher. **55**

Les curseurs (13/22)

Attributs d'un curseur explicite

Attribut	Type	Description
%ISOPEN	Booléen	Prend la valeur <code>TRUE</code> si le curseur est ouvert
%NOTFOUND	Booléen	Prend la valeur <code>TRUE</code> si la dernière extraction (fetch) ne renvoie pas de ligne
%FOUND	Booléen	Prend la valeur <code>TRUE</code> si la dernière extraction renvoie une ligne ; complément de %NOTFOUND
%ROWCOUNT	Nombre	Prend la valeur correspondant au nombre total de lignes renvoyées jusqu'à présent

Les curseurs (14/22)

Attribut %ISOPEN

- Les lignes ne peuvent être extraites que si le curseur est ouvert.
- L'attribut de curseur `%ISOPEN` sert à déterminer si le curseur est ouvert.
- `%ISOPEN` renvoie l'état du curseur à `TRUE` s'il est ouvert et `FALSE` s'il est fermé.

Exemple

- On peut tester si un curseur est ouvert avant de commencer à extraire les lignes

```
IF NOT emp_cursor%ISOPEN THEN
    OPEN emp_cursor;
END IF;
LOOP
    FETCH emp_cursor...
```

Les curseurs (15/22)

Attribut %ROWCOUNT

- Sert à :
- Extraire un nombre exact de lignes
- Extraire (fetch) les lignes avec une boucle et déterminer dans quels cas la sortie de la boucle doit s'effectuer

Exemple

- Ecrire un programme pl/sql permettant d'extraire et d'afficher les 4 premiers employés appartenants au service 10 s'ils existent sinon afficher uniquement ceux qui existent.
- Rq! Il faut faire attention à la condition de sortie de la boucle.

Les curseurs (16/22)

Utilisation de sous interrogation dans la boucle FOR du curseur

On peut ne pas déclarer un curseur dans la boucle for et utiliser à la place directement une sous interrogation

Syntaxe

```
FOR nom_enregistrement IN (Requête) LOOP  
    instruction1;  
    instruction2;  
    . . .  
END LOOP;
```

Exemple

- Extraire et afficher les personnes travaillant au service 20.

Les curseurs (17/22)

Curseurs avec paramètres

Permettent de transmettre des paramètres au curseur au moment de son ouverture et de l'exécution de l'interrogation.

Ouvrir un curseur explicite à plusieurs reprises, en renvoyant un ensemble actif différent à chaque fois.

Syntaxe

```
CURSOR nom_curseur [(nom_parametre type, ...)]  
IS select_statement;
```

```
OPEN      nom_curseur (val_parametre,.....) ;  
          instructions;  
CLOSE nom curseur;
```

Les curseurs (18/22)

Exemple

- *Extraire et afficher en fonction du numéro de service*
- le numéro du service et le nombre total des personnes y travaillant pour le service10.
- Le numéro du service et la moyenne des salaires des personnes y travaillant pour le service30,
- Le numéro du service et la somme totale des salaires des personnes travaillant au service 60.

Les curseurs (19/22)

Clause FOR UPDATE

- *Lorsque plusieurs sessions pour une même base de données sont ouvertes, les lignes d'une table particulière peuvent être mises à jour par plusieurs utilisateurs après l'ouverture du curseur.*
- *Les données mises à jour ne peuvent être vues que lorsque le curseur est ouvert de nouveau.*
- *Il est donc préférable de placer des verrous sur les lignes avant de les mettre à jour ou de les supprimer.*
- *Le verrouillage des lignes se fait avec la clause `FOR UPDATE` dans l'interrogation du curseur.*

Les curseurs (20/22)

Syntaxe

```
SELECT      ...  
FROM        ...  
FOR UPDATE [OF column_reference] [NOWAIT | WAIT n];
```

- Le verrouillage explicite sert à interdire l'accès aux autres sessions pendant la durée d'une transaction.
- Le verrouillage des lignes se fait avant la mise à jour ou la suppression.

L'instruction **SELECT ... FOR UPDATE** identifie les lignes qui seront mises à jour ou supprimées, puis verrouille chaque ligne de l'ensemble de résultats.

Cela s'avère utile lorsqu'une mise à jour est basée sur les valeurs existantes d'une ligne. Dans ce cas, il faut s'assurer que cette ligne n'est pas modifiée par une autre session avant la mise à jour.

Les curseurs (21/22)

- Le mot-clé facultatif **NOWAIT** indique au serveur Oracle de ne pas attendre si les lignes demandées soient verrouillées par un autre utilisateur.
- Le programme reprend immédiatement le contrôle ; il peut ainsi effectuer d'autres travaux avant de réessayer d'obtenir le verrouillage.
- Si vous omettez le mot-clé **NOWAIT**, le serveur Oracle attend que les lignes soient disponibles mais il peut attendre indéfiniment.
- Si les lignes sont verrouillées par une autre session et que le mot-clé **NOWAIT** est indiqué, l'ouverture du curseur provoque une erreur (il faut essayer de le rouvrir ultérieurement).
- On peut utiliser **WAIT** plutôt que **NOWAIT** et indiquer le nombre de secondes pendant lesquelles attendre et vérifier que les lignes sont déverrouillées. Si les lignes sont toujours verrouillées après n secondes, une erreur est renvoyée.

Les curseurs (22/22)

Clause WHERE CURRENT OF

La clause **WHERE CURRENT OF** est utilisée conjointement avec la clause **FOR UPDATE** afin de faire référence à la ligne en cours dans un curseur explicite.

La clause **WHERE CURRENT OF** est utilisée dans l'instruction **UPDATE** ou **DELETE**, tandis que la clause **FOR UPDATE** est définie dans la **déclaration du curseur**.

La clause **FOR UPDATE** peut être incluse dans l'interrogation du curseur afin que les lignes soient verrouillées lors de l'ouverture (**OPEN**).

Exemple

- *Mettre à jour les salaires des personnes du service 10 en les augmentant de 10%.*